

05 — Latência vs Bandwidth

Pré-requisito: [04-memory-hierarchy.md](#)

Este é, honestamente, o arquivo mais importante do módulo. A confusão entre essas duas coisas é a fonte de mais raciocínio errado sobre performance do que qualquer outro conceito nesta fase.

1. As duas perguntas, que parecem a mesma coisa mas não são

- **Latência:** quanto tempo leva **UMA** operação, do início ao fim, sozinha?
- **Bandwidth (throughput):** quantas operações terminam **por unidade de tempo**, em regime contínuo, com muitas acontecendo?

Você já viu essa distinção duas vezes neste módulo, com nomes diferentes: a linha de montagem (pergunta rápida do arquivo 01 — 4 ciclos pra UM produto, 1 produto por ciclo em regime). Vamos agora tratar isso como o conceito central, formalmente, porque ele reaparece sem parar daqui pra frente.

2. A analogia que resolve a confusão de vez: a autoestrada

Imagina uma autoestrada entre duas cidades.

- **Latência:** quanto tempo um carro **sozinho** leva pra ir de uma cidade à outra — digamos, 1 hora, não importa quantas pistas a estrada tem.
- **Bandwidth:** quantos carros **chegam por hora**, em regime contínuo, se a estrada estiver lotada de carros o tempo todo.

Uma estrada de 1 pista e uma de 8 pistas têm a **mesma latência** (ainda leva 1 hora pra um carro sozinho atravessar) — mas a de 8 pistas tem **8x mais bandwidth** (8x mais carros chegando por hora, no regime cheio). Adicionar pistas não faz nenhum carro individual chegar mais rápido — faz **mais carros** chegarem no mesmo período.

PERGUNTA RÁPIDA 1: se você é o único carro na estrada (nenhum outro carro), o número de pistas importa pro seu tempo de viagem?

► Resposta

3. Como você "esconde" latência: paralelismo de requisição

A forma de realmente aproveitar bandwidth alta não é "esperar menos por requisição" — é **ter várias requisições em voo ao mesmo tempo**, de forma que, enquanto uma está "viajando" (latência em andamento), outras já estão sendo despachadas.

Uma requisição de cada vez (não aproveita bandwidth):

Req 1: [espera 100ns].....[chega]

Req 2: [espera

100ns].....[chega]

Tempo total pra 2 requisições: ~200ns

Requisições sobrepostas (aproveitando bandwidth disponível):

```
Req 1: [espera 100ns].....[chega]
Req 2:  [espera 100ns].....[chega]    <- começou quase
      junto
Req 3:  [espera 100ns].....[chega]
Tempo total pra 3 requisições sobrepostas: ~100ns + um pouquinho, não 300ns
```

Isso é exatamente o que pipelining (arquivo 01) faz para instruções, e é exatamente o que hardware de memória faz para requisições de leitura: a latência de uma requisição individual não muda, mas se você tem muitas em voo, o **throughput** agregado se aproxima do limite de bandwidth, não da soma das latências individuais.

4. Aplicando isso a memória, concretamente

RAM tem uma latência de ~100ns por acesso **isolado**. Mas a bandwidth sustentável de uma RAM moderna é de dezenas de GB/s. Esses dois números não são inconsistentes — eles medem coisas diferentes:

- Se você faz **um** acesso, espera o resultado, faz **outro** acesso, espera de novo — você está pagando 100ns por acesso, sequencialmente. Isso é *latency-bound*.
- Se você tem muitos acessos em voo simultaneamente (o hardware de memória processa vários pedidos concorrentes), o throughput agregado se aproxima da bandwidth de pico. Isso é *bandwidth-bound*.

Código que percorre um array sequencialmente, onde o hardware consegue prever o próximo endereço e "pré-buscar" (prefetch) antes de você pedir, se aproxima do regime de bandwidth. Código que segue ponteiros aleatórios (uma lista encadeada espalhada pela memória, por exemplo) fica preso no regime de latência — cada acesso depende do resultado do anterior pra saber onde ir, então não há como sobrepor.

PERGUNTA RÁPIDA 2: por que percorrer uma lista encadeada (`while (node) node = node->next;`) é um dos padrões de acesso mais lentos que existem, mesmo que o total de dados seja pequeno?

► Resposta

5. Por que isso é a base do resto do módulo

A pergunta central deste módulo inteiro — "essa operação é limitada por cálculo ou por memória?" — só faz sentido depois de você separar latência de bandwidth. "Limitado por memória" quase sempre quer dizer **limitado pela bandwidth de memória disponível**, não pela latência de um acesso isolado — porque hardware moderno (CPU e GPU) é desenhado especificamente para sobrepor muitas requisições e se aproximar do regime de bandwidth, sempre que o padrão de acesso permite.

6. Little's Law: quantificando "quantas requisições em voo eu preciso"

Os arquivos anteriores explicaram **qualitativamente** que você precisa de várias requisições sobrepostas pra aproximar bandwidth. Little's Law dá a **fórmula exata** de quantas.

Little's Law vem originalmente da teoria de filas (pensamento sobre pessoas numa fila de banco, não sobre memória), mas se aplica exatamente igual aqui:

$$L = \lambda \times W$$

L = número médio de itens "no sistema" (em voo, em processamento)

λ = taxa de chegada (throughput – quantos itens entram por unidade de tempo)

W = tempo médio que cada item passa no sistema (latência)

Traduzindo pro vocabulário deste módulo:

Concorrência necessária $\approx$ Throughput desejado $\times$ Latência por requisição

7. O exemplo que conecta isso com número real

Suponha que você quer **saturar** uma bandwidth de memória de 25 GB/s, e cada requisição de memória individual tem latência de 100ns, movendo 64 bytes (uma cache line).

Throughput desejado (em requisições/segundo):

25 GB/s / 64 bytes por requisição = ~390 milhões de requisições/segundo

Concorrência necessária = throughput $\times$ latência:

390.000.000 requisições/s $\times$ 0,0000001 s (100ns) = 39 requisições em voo, ao mesmo tempo

Esse é o número central: pra saturar 25 GB/s com requisições de 100ns de latência cada, você precisa de **~39 requisições pendentes simultaneamente**, o tempo todo. Se seu código só consegue ter 1 requisição em voo por vez (o caso do pointer chasing dependente, arquivo 11), você nunca chega nem perto — está preso no regime de latência, não de bandwidth, exatamente como o arquivo 05 (este arquivo) já descreveu qualitativamente. Little's Law só transformou isso numa conta exata.

PERGUNTA RÁPIDA 2: usando os números que você mediu de verdade nos exemplos deste módulo ([01-latency-benchmark](#) pra latência, [02-bandwidth-benchmark](#) pra bandwidth), calcule a concorrência necessária pro **seu próprio hardware**. O número que você calculou é grande ou pequeno comparado a "1 requisição por vez"?

► Como calcular com seus próprios números

8. Um limite importante deste modelo (não é lei física perfeita)

Little's Law é uma **aproximação útil pra desenvolver intuição**, não uma descrição completa e exata de como CPU ou GPU escondem latência de memória na prática. Hardware real tem filas finitas (um número máximo de requisições em voo que ele consegue de fato gerenciar — não é ilimitado), efeitos de contenção quando múltiplos núcleos/threads competem pelo mesmo canal de memória, e comportamento que varia com o padrão de acesso específico, não só a média. Trate Little's Law como uma primeira aproximação — "essa é, grosso modo, a ordem de grandeza de concorrência que eu preciso" — não como um número que prevê performance exata.

Uma ideia importante pra já plantar aqui, antes da Phase 2 desenvolver isso a fundo: alta occupancy (muitos warps disponíveis pra troca) não significa automaticamente alta performance. Occupancy alta é o que **permite** concorrência suficiente pra esconder latência — mas se o padrão de acesso dentro de cada thread ainda for ruim (pouca localidade, muito conflict miss, etc.), ter muitos warps disponíveis não resolve isso sozinho. Little's Law te diz "quanta concorrência você **precisa**" — não garante que tê-la resolve todos os outros problemas de performance ao mesmo tempo. Essa distinção vai ficar mais concreta quando você estudar occupancy de verdade na Phase 2.

9. Por que isso importa para CUDA

Isso é, sem exagero, o conceito mais transferível de todo o módulo pra GPU. Uma GPU tem latência de acesso à memória global **pior** que uma CPU (centenas de ciclos) — mas compensa isso tendo **milhares de threads** capazes de ter requisições em voo simultaneamente. Quando um warp fica esperando um acesso à memória, o Streaming Multiprocessor troca pra executar outro warp que já está pronto — escondendo a latência do primeiro atrás do trabalho útil do segundo. Isso se chama **latency hiding**, e é literalmente a mesma ideia da autoestrada de várias pistas: a GPU não torna cada acesso individual mais rápido, ela mantém tantas requisições "em voo" que o throughput agregado se aproxima do limite de bandwidth, apesar da latência individual ruim.

10. Erros comuns

- Otimizar pra "menos operações" quando na verdade o problema é falta de sobreposição — às vezes o número certo de operações já está lá, só precisa ser reorganizado pra permitir paralelismo de requisição.
- Achar que bandwidth alta ajuda automaticamente. Só ajuda se seu padrão de acesso **permite** múltiplas requisições concorrentes (acesso previsível, sem dependência entre requisições).
- Confundir "essa operação é rápida" com "essa operação usa bem a bandwidth disponível" — uma operação pode ser rápida em latência absoluta e ainda estar longe do teto de bandwidth, se não houver paralelismo suficiente de requisição.
- Tratar Little's Law como garantia de performance, não como estimativa de ordem de grandeza — hardware real tem limites de fila, contenção e comportamento específico de padrão de acesso que o modelo simples não captura (seção 8).
- Achar que "muita concorrência disponível" (occupancy alta, na terminologia que a Phase 2 vai usar) já garante performance alta sozinho — ela só resolve o problema de "esconder latência"; um padrão de acesso ruim dentro de cada thread continua sendo um problema separado.

PREDICT BEFORE RUNNING (conceitual)

Um SSD tem latência de ~100 microssegundos por leitura isolada, mas bandwidth sustentável de ~3 GB/s quando várias leituras acontecem em paralelo.

1. Se você lê 1000 blocos pequenos, um de cada vez, esperando cada um terminar antes de pedir o próximo, o tempo total é dominado pela latência ou pela bandwidth?
2. Se você pede os mesmos 1000 blocos, mas dispara todos os pedidos "de uma vez" (I/O assíncrono, deixando o sistema operacional/controlador do SSD processar vários em paralelo), o que muda?
3. Isso é análogo a qual conceito de CUDA que você já ouviu falar hoje, mesmo sem profundidade ainda (dica: streams, ou lançar múltiplos kernels)?

4. Usando Little's Law: se você quer atingir a bandwidth de 3 GB/s desse SSD, com blocos de 4 KB por requisição e latência de $100\mu\text{s}$ cada, quantas requisições precisam estar em voo simultaneamente?

► Explicação